

TECHNICAL REPORT

Energy-Conscious Autonomous Mobile Robot

Ghada Ammar

2025

Table des matières

1	Chapitre 1 : Organisme d'accueil et travail demandé	7
1.1	Introduction :	7
1.2	Présentation de l'entreprise :	7
1.2.1	Produits et services :	7
1.3	Problématique traitée et travail demandé :	7
1.4	Méthodologie et Architecture Système	8
1.4.1	Phase I : Établissement d'une Base de Simulation et de Navigation Fiable	8
1.4.2	Phase II : Conception du Modèle de Conscience Énergétique par Apprentissage Supervisé	9
1.4.3	Phase III : Intégration du Modèle et Architecture Décisionnelle .	9
1.4.4	Phase IV : Réalisation de la Boucle d'Autonomie Complète	10
1.5	Chronogramme des tâches menées durant le stage :	10
1.6	Conclusion : Principales observations et constations :	11
2	Chapitre 2 : Analyse des besoins et conception :	12
2.1	Introduction :	12
2.2	Étude de l'Existant :	12
2.2.1	Systèmes Basés sur des Règles Statiques :	12
2.2.2	Systèmes de Navigation avec Docking Intégré :	12
2.2.3	Technologies de Perception pour le Docking :	13
2.3	Spécification des Besoins :	13
2.3.1	Besoins Fonctionnels :	13
2.3.2	Besoins Non Fonctionnels :	14
2.4	Conception de l'Architecture Système :	15
2.4.1	Vue d'Ensemble de l'Architecture :	15
2.4.2	Interfaces de communication	17
2.5	Conception du Module d'Intelligence Décisionnelle :	18
2.5.1	Modélisation du Problème :	18
2.5.2	Ingénierie des Caractéristiques (Feature Engineering) :	18
2.5.3	Logique de Contrôle de l'Agent :	18

2.5.4	Pipeline d'Apprentissage Automatique :	20
2.6	Conclusion :	22
3	Chapitre 3 : Réalisation :	23
3.1	Introduction :	23
3.2	Environnements de travail :	23
3.2.1	Environnement matériel :	23
3.3	Choix technologiques :	23
3.3.1	ROS2 (Robotic Operating System) :	23
3.3.2	Rviz :	26
3.3.3	Gazebo :	26
3.4	Fonctionnalités réalisées :	27
3.5	Conclusion :	36

Liste des figures

1	Diagramme de l'architecture logicielle du système.	16
2	Logigramme de l'Agent Décisionnel Autonome.	19
3	Pipeline d'Apprentissage Automatique	21
4	Exemple de communication en mode Topic	24
5	Communication en mode Service	25
6	Différents modes de communication	25
7	Logo Rviz	26
8	Logo Gazebo	26
9	Environnement de simulation Gazebo	27
10	Architecture simplifiée de la pile Nav2 et interactions principales	29
11	Docking en simulation	30
12	Extrait du jeu de données d'entraînement	32
13	Processus d'annotation	33
14	Matrice de confusion du modèle sur le jeu de test.	33
15	Importance des caractéristiques pour la prédiction du modèle.	34

Liste des tableaux

1	Chronogramme des tâches du stage	11
2	Interfaces de communication du système ROS 2	17
3	Environnement matériel	23
4	Rapport de classification détaillé par classe.	34

Introduction Générale

L'avènement des agents intelligents constitue une avancée majeure dans le domaine de la robotique mobile, marquant la transition de simples automates exécutant des trajectoires prédéfinies vers des systèmes véritablement autonomes. Au cœur de cette autonomie se trouve la navigation autonome, une capacité fondamentale qui permet à un robot de se localiser, de cartographier et de se déplacer de manière efficiente dans des environnements complexes et dynamiques. Des cadres de développement comme le Robot Operating System (ROS 2) et sa suite de navigation Nav2 ont standardisé les outils pour la planification de chemin et l'évitement d'obstacles, rendant la navigation robuste plus accessible. Cependant, la véritable autonomie transcende la simple mobilité. Elle exige une persistance opérationnelle sur le long terme, ce qui soulève une problématique critique : la gestion de l'énergie. Un agent intelligent ne doit pas seulement savoir où aller, mais aussi quand et si il a les ressources nécessaires pour accomplir sa mission et survivre. Cette prise de conscience, ou "Energy Awareness", est ce qui distingue un robot autonome d'un robot automatisé. Il doit être capable d'évaluer son état énergétique interne, d'estimer le coût de ses actions futures et de prendre des décisions stratégiques pour garantir sa propre pérennité, notamment en initiant un retour à sa station de charge avant l'épuisement de ses batteries. Ce projet se situe précisément à l'intersection de la navigation autonome et de la conscience énergétique. L'objectif est de développer un agent mobile qui ne se contente pas de suivre des ordres, mais qui possède une couche d'intelligence décisionnelle lui permettant de gérer son cycle de vie énergétique. Pour ce faire, nous avons exploité l'écosystème ROS 2 pour mettre en place une pile de navigation complète, incluant un système de docking de précision basé sur des marqueurs visuels AprilTag pour assurer une recharge fiable. Ce rapport s'articule autour de trois chapitres principaux, précédés d'une introduction générale. Il débute par une présentation de l'entreprise et de son domaine d'application, puis propose un aperçu de l'état de l'art en robotique ainsi qu'une description du robot et de ses capteurs. Enfin, il expose la réalisation du projet, les tests effectués et les résultats obtenus lors des simulations.

Mots-clés : Conscience Énergétique (Energy Awareness), Navigation Autonome, Machine Learning, Gestion d'Énergie, Docking de Précision.

1 Chapitre 1 : Organisme d'accueil et travail demandé

1.1 Introduction :

Dans ce chapitre, je propose de présenter l'entreprise d'accueil . De plus, Je vais aborder le travail qui m'a été confié durant le stage. Enfin, je détaillerai la méthodologie de travail adoptée pendant le stage, et les tâches seront illustrées par un chronogramme pour couvrir toute la période du stage.

1.2 Présentation de l'entreprise :

La société IBACONSEIL, créée en juillet 2023, est une société unipersonnelle à responsabilité limitée, au capital de 2 000 dinars, ayant son siège social au 000 Cité Ettamir Imm.Amor , 4000, Sousse, TUNISIE, identifiée par le numéro unique N° 1823793E, et représentée par sa Gérante, Madame Ines BEN ABDALLAH.

1.2.1 Produits et services :

IBACONSEIL se spécialise dans la gestion de projets de développement informatique indépendant et propose des services de consultation en innovation ainsi qu'en numérisation des processus et métiers pour des clients. Avec une expertise en transformation digitale, la société optimise l'efficacité opérationnelle. Les services incluent le développement de solutions personnalisées pour répondre aux besoins spécifiques des clients, la mise en œuvre de stratégies informatiques novatrices pour stimuler la croissance, et le leadership dans la conception et la mise en place de solutions technologiques efficaces

1.3 Problématique traitée et travail demandé :

La problématique centrale de ce projet est de dépasser la simple navigation autonome pour atteindre une persistance opérationnelle à long terme pour un robot mobile. Un robot capable de naviguer de point A à point B est fonctionnel, mais il reste limité par son autonomie énergétique. Sans une gestion intelligente de ses ressources, il

est voué à l'échec, incapable de garantir sa disponibilité pour des missions prolongées. Le travail demandé consiste donc à concevoir et implémenter une couche d'intelligence décisionnelle qui confère au robot une conscience énergétique. L'objectif n'est pas seulement de déclencher une recharge à un seuil de batterie fixe, mais de développer un système prédictif capable de répondre aux questions suivantes : Étant donné ma position actuelle et ma mission, ai-je suffisamment d'énergie pour atteindre mon objectif ET revenir à la station de charge en toute sécurité ? Quel est le moment optimal pour interrompre ma tâche actuelle afin de prioriser ma survie énergétique ? Pour répondre à cette problématique, le projet vise à : Mettre en place un système de docking de précision basé sur des marqueurs visuels (AprilTag) comme solution de recharge fiable. Développer un modèle de Machine Learning capable de prédire la nécessité d'une recharge en se basant sur l'état du robot (niveau de batterie, distance au dock). Intégrer ce modèle dans un agent décisionnel autonome capable d'arbitrer entre les ordres de mission et l'impératif de recharge. Valider une boucle d'autonomie complète (mission → recharge → reprise de mission) dans un environnement de simulation haute-fidélité (Gazebo).

1.4 Méthodologie et Architecture Système

La réalisation de cet agent mobile autonome, doté d'une conscience énergétique, a reposé sur une méthodologie de développement incrémentale et modulaire. Cette approche a permis de valider chaque sous-système de manière indépendante avant son intégration dans l'architecture globale, garantissant ainsi la robustesse et la traçabilité du système final. La démarche s'est articulée en quatre phases stratégiques.

1.4.1 Phase I : Établissement d'une Base de Simulation et de Navigation Fiable

Le fondement de tout développement en robotique réside dans un environnement de test robuste. Nous avons initié le projet en configurant un écosystème de simulation complet sous **ROS 2 Jazzy**.

- **Modélisation de l'Environnement Physique** : Le simulateur **Gazebo** a été employé pour créer un jumeau numérique du robot et de son espace d'opération, permettant ainsi un cycle de développement et de débogage rapide et itératif.
- **Implémentation de la Pile de Navigation Nav2** : Le framework **Nav2** a

été déployé pour conférer au robot ses capacités de navigation fondamentales. Le module de localisation **AMCL** a été configuré pour l'estimation de pose en temps réel, tandis que les serveurs de planification et de contrôle ont été paramétrés pour assurer une navigation autonome.

- **Mise en Œuvre du Docking de Précision** : Un **AprilTag** a été positionné sur la station de charge, servant de repère visuel. Le serveur de docking de Nav2 a été configuré pour exploiter les données de pose issues de ce marqueur, permettant ainsi des manœuvres terminales fiables.

1.4.2 Phase II : Conception du Modèle de Conscience Énergétique par Apprentissage Supervisé

Cette phase constitue le cœur de l'innovation du projet : doter le robot de la capacité d'anticiper ses propres besoins énergétiques.

- **Modélisation de la Dynamique Énergétique** : Un nœud de simulation de batterie a été développé pour modéliser la décharge en fonction de l'effort cinématique du robot, fournissant ainsi un état de charge réaliste et dynamique.
- **Constitution d'un Jeu de Données d'Entraînement** : Une campagne de collecte de données a été menée en environnement simulé, capturant des vecteurs caractéristiques tels que le niveau de batterie, la distance géodésique au dock et le coût énergétique estimé du trajet retour.
- **Annotation Automatisée (Labeling)** : Chaque vecteur a été annoté par une « décision experte » (EXPLORE ou DOCK), selon une heuristique basée sur l'autonomie résiduelle du robot.
- **Entraînement et Validation** : Le modèle a été entraîné via l'algorithme **XG-Boost**, reconnu pour sa précision et son efficacité. Le pipeline a ensuite été sérialisé pour son déploiement opérationnel.

1.4.3 Phase III : Intégration du Modèle et Architecture Décisionnelle

L'intelligence prédictive a été intégrée au sein de l'architecture de contrôle.

- **Développement de l'Agent Décisionnel** : Un nœud ROS 2 central, conçu comme le « cerveau exécutif », charge le modèle de Machine Learning.
- **Implémentation de la Boucle de Décision** : À une fréquence définie, le nœud

perçoit l'état du robot, consulte le modèle pour obtenir une prédiction et agit en conséquence. Si la décision est **DOCK**, il interrompt la navigation et initie le docking ; sinon, il poursuit la mission.

1.4.4 Phase IV : Réalisation de la Boucle d'Autonomie Complète

La dernière phase a consisté à fermer la boucle pour assurer une autonomie complète.

- **Gestion du Cycle de Recharge** : Le simulateur de batterie augmente l'état de charge lorsque le robot est docké.
- **Automatisation du Détachement** : Une fois la charge complète atteinte, l'agent déclenche la manœuvre de retrait.
- **Réinitialisation du Cycle Comportemental** : Le robot revient à un état nominal, prêt à accepter de nouvelles missions. Le cycle complet « mission → décision de recharge → docking → recharge → détachement → reprise de mission » est ainsi bouclé.

1.5 Chronogramme des tâches menées durant le stage :

Pour bien organiser le projet, j'ai préparé un plan des différentes étapes. Voici ci-dessous le chronogramme des tâches prévues durant le stage. Ce plan permet de suivre facilement le déroulement du projet.

	Juillet														Août									
Tâche	14	15	16	17	18	21	22	23	24	28	29	30	31	1	4	5	6	7	8	11	12	13		
Établissement d'une Base de Simulation et de Navigation fiable																								
Conception du Modèle de Conscience énergétique par Apprentissage supervisé																								
Intégration du Modèle et Architecture Décisionnelle																								
Réalisation de la Boucle d'Autonomie Complet																								

TABLE 1 – Chronogramme des tâches du stage

1.6 Conclusion : Principales observations et constations :

Dans ce chapitre, j'ai présenté brièvement l'entreprise dans laquelle j'ai réalisé mon stage. J'ai ensuite planifié les tâches nécessaires pour aboutir au travail demandé par l'ingénieur. Dans la suite du rapport, je présenterai les différentes parties du travail réalisé

2 Chapitre 2 : Analyse des besoins et conception :

2.1 Introduction :

Ce chapitre traduit la problématique d'un agent mobile à conscience énergétique en exigences fonctionnelles et non fonctionnelles, puis présente la conception de l'architecture logicielle sous ROS 2 et le développement du module d'intelligence décisionnelle, élément central du système.

2.2 Étude de l'Existant :

La gestion autonome de l'énergie pour les robots mobiles est un domaine de recherche actif, où plusieurs approches coexistent.

2.2.1 Systèmes Basés sur des Règles Statiques :

L'approche la plus courante consiste à utiliser des seuils fixes. Un robot est programmé pour initier une procédure de recharge dès que son niveau de batterie passe en dessous d'un seuil prédéfini (par exemple, 20%).

- **Avantages :** Simplicité d'implémentation, faible coût de calcul.
- **Critique et Limites :** Cette méthode manque de conscience contextuelle. Un robot peut se retrouver dans une situation où, bien qu'ayant 25% de batterie, il est trop éloigné du dock pour y retourner à temps. Cette approche est donc rigide et peu fiable dans des environnements vastes ou complexes.

2.2.2 Systèmes de Navigation avec Docking Intégré :

Des solutions avancées, comme le serveur de docking de Nav2, offrent des mécanismes robustes pour les manœuvres de recharge de précision. Ces systèmes utilisent souvent des marqueurs visuels pour garantir un alignement parfait.

- **Avantages :** Grande fiabilité des manœuvres de connexion physique.
- **Critique et Limites :** Ces systèmes fournissent l'outil pour exécuter le docking, mais ne répondent pas à la question fondamentale de « quand » l'initier. La décision de lancer l'action de docking reste à la charge d'une logique externe.

2.2.3 Technologies de Perception pour le Docking :

Pour assurer un alignement précis, plusieurs technologies de perception sont utilisées. Les marqueurs visuels fiduciels sont particulièrement populaires pour leur robustesse et leur faible coût.

- **Marqueurs ArUco :** Largement utilisés, ils sont faciles à détecter. Cependant, ils peuvent être moins robustes aux occlusions partielles et aux conditions d'éclairage difficiles.
- **AprilTags :** Une évolution des marqueurs fiduciels, conçus pour être plus robustes aux variations de luminosité et aux angles de vue complexes. Ils offrent une détection plus rapide et un taux de faux positifs plus faible, ce qui est critique pour une application de recharge fiable.
- **Positionnement de notre projet :** Notre travail se positionne comme une synthèse intelligente des approches existantes. Nous utilisons la robustesse du docking de Nav2 et la précision des AprilTags, mais nous remplaçons la logique de décision statique par un agent décisionnel intelligent basé sur l'apprentissage automatique, comblant ainsi le manque de conscience contextuelle des systèmes actuels.

2.3 Spécification des Besoins :

La conception d'un système fiable et pertinent commence par une définition claire et non ambiguë de ses objectifs et de ses contraintes de performance. Cette section formalise le cahier des charges de notre projet, en distinguant les capacités attendues du système (besoins fonctionnels) de ses critères de qualité (besoins non fonctionnels).

2.3.1 Besoins Fonctionnels :

Les besoins fonctionnels décrivent les actions et les capacités que le système doit impérativement accomplir pour être considéré comme une réussite.

- **Navigation Autonome et Localisation.** Le système doit permettre au robot de se localiser en temps réel sur une carte préexistante et de naviguer de manière autonome vers un point de consigne, en planifiant des trajectoires qui évitent les obstacles statiques et dynamiques.

- **Gestion du Cycle de Vie Énergétique.** Le système doit intégrer une modélisation dynamique de l'énergie, simulant la décharge de la batterie en fonction de l'activité motrice et sa recharge lorsqu'il est connecté à sa station.
- **Manœuvre de Docking de Précision.** Le robot doit être capable d'exécuter une manœuvre d'amarrage autonome et de haute précision avec sa station de charge, en utilisant un marqueur visuel AprilTag comme référence pour garantir un contact physique fiable.
- **Prise de Décision Basée sur l'Apprentissage.** L'agent doit posséder un module décisionnel basé sur un modèle de Machine Learning pré-entraîné, capable de déterminer l'opportunité d'une recharge en fonction de l'état courant du système.
- **Arbitrage et Priorisation des Tâches.** Le module décisionnel doit avoir l'autorité d'interrompre une tâche de navigation active (initiée par un opérateur) si une recharge est jugée prioritaire pour la survie du robot.
- **Boucle d'Autonomie Complète.** Le système doit démontrer un cycle opérationnel complet et ininterrompu : exécution d'une mission, décision autonome de recharge, docking, charge effective, manœuvre de retrait autonome, et retour à un état de disponibilité pour de nouvelles missions.

2.3.2 Besoins Non Fonctionnels :

Les besoins non fonctionnels définissent les critères de qualité, les contraintes de performance et les attributs opérationnels du système.

- **Fiabilité Décisionnelle.** Le modèle prédictif doit atteindre une exactitude (accuracy) supérieure à 95% sur un jeu de données de test représentatif, afin de minimiser les risques de défaillance énergétique (panne sèche) ou de recharges inutiles.
- **Modularité et Maintenabilité.** L'architecture logicielle doit être fortement modulaire, basée sur des nœuds ROS 2 indépendants avec des interfaces de communication (topics, actions) clairement définies. Ce découplage est essentiel pour faciliter les tests unitaires, la maintenance et les évolutions futures.
- **Réactivité du Système.** La boucle de décision de l'agent doit s'exécuter à une fréquence d'au moins 0.2 Hz (toutes les 5 secondes) pour garantir une réactivité suffisante aux changements d'état de l'environnement et du robot.

- **Standardisation et Interopérabilité.** La solution doit être exclusivement développée sur le framework ROS 2 Jazzy et s'appuyer sur des paquets standards de l'écosystème (Nav2, Gazebo, RViz) pour assurer sa portabilité et sa compatibilité.

2.4 Conception de l'Architecture Système :

Pour répondre à l'ensemble de ces spécifications, une architecture distribuée et modulaire a été conçue. Cette section détaille la structure logique du système et les interactions entre ses composants.

2.4.1 Vue d'Ensemble de l'Architecture :

Le choix s'est porté sur une architecture logicielle décentralisée sous ROS 2, où chaque compétence fondamentale est encapsulée dans un nœud dédié. Cette approche favorise la séparation des préoccupations (Separation of Concerns), améliorant ainsi la robustesse, la testabilité et la maintenabilité du système.

Le diagramme ci-dessous illustre l'organisation de ces pôles fonctionnels ainsi que les flux de données qui les relient.

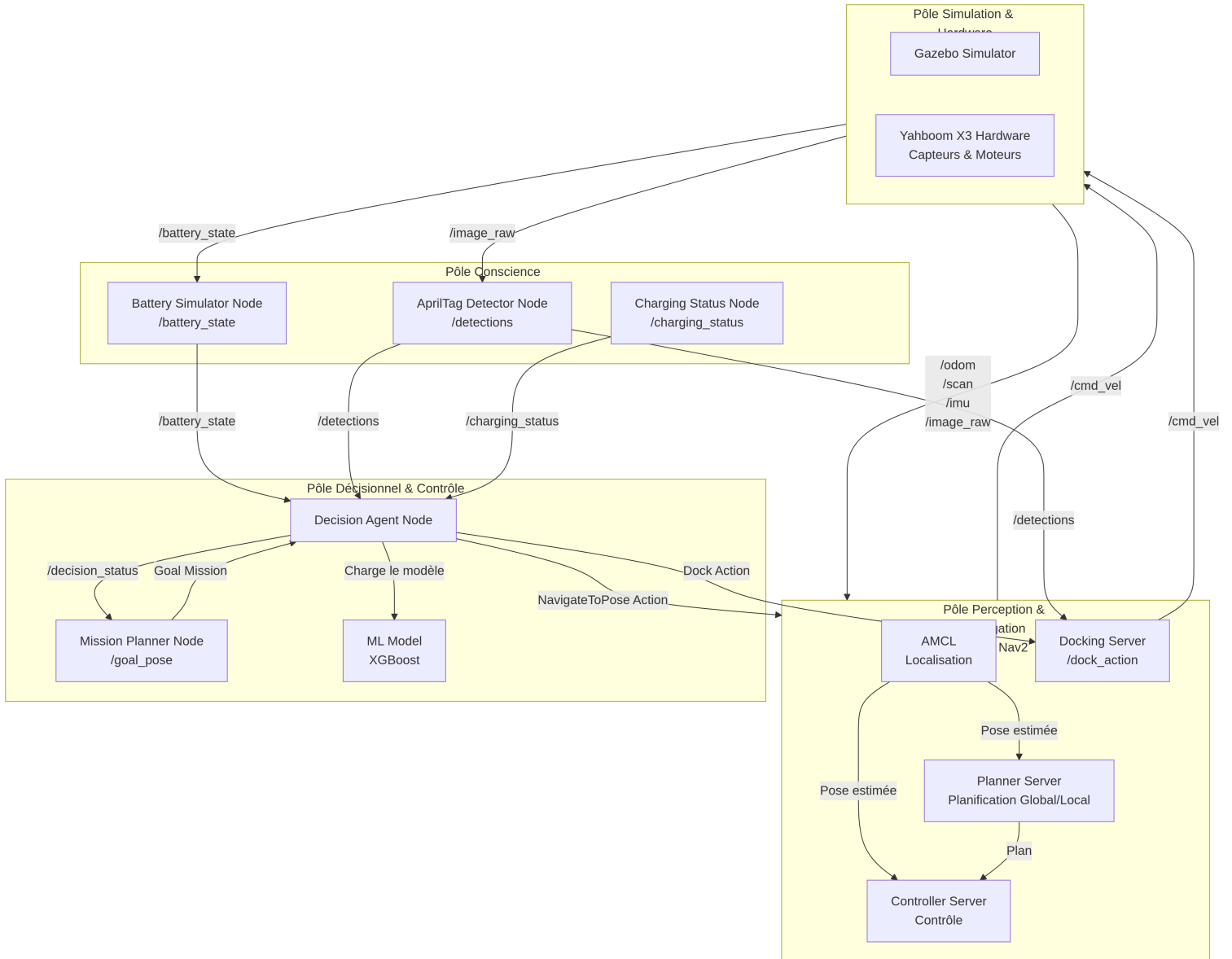


FIGURE 1 – Diagramme de l'architecture logicielle du système.

L'architecture s'articule autour de quatre pôles fonctionnels principaux :

- **Le Pôle Simulation & Hardware** : Il fournit une représentation physique réaliste du robot et de son environnement. Il publie les données brutes des capteurs (scan laser, images de caméra, IMU) ainsi que l'odométrie brute, et est conçu pour que le simulateur Gazebo puisse être remplacé par le matériel réel.
- **Le Pôle Perception & Navigation** : Il consomme les données des capteurs et l'odométrie pour fournir des services de haut niveau. AMCL assure la localisation, tandis que les serveurs de planification et de contrôle de Nav2 gèrent les déplacements. Le serveur de docking est un composant clé, orchestrant les

manœuvres finales de recharge.

- **Le Pôle Conscience** : Il s’agit de l’ensemble des nœuds développés pour donner au robot sa conscience de l’état. Un nœud de simulation de batterie publie en continu le niveau d’énergie, tandis qu’un nœud de statut de charge agit comme un verrou binaire pour synchroniser l’état de charge entre les différents modules.
- **Le Pôle Décisionnel & Contrôle (le ”Cerveau”)** : C’est le cœur de notre système. Le nœud décisionnel s’abonne aux informations de localisation et d’énergie. Il utilise ces données pour interroger le modèle de Machine Learning et prendre une décision stratégique. En fonction du résultat, il publie des commandes sur les serveurs d’action de Nav2.

2.4.2 Interfaces de communication

La communication entre ces pôles est assurée par les mécanismes standards de ROS

2. Les interfaces principales sont définies dans le tableau 2 ci-dessous.

Interface	Type	Description	Émetteur	Récepteur
/amcl_pose	Topic	Fournit la pose estimée du robot sur la carte.	Nav2 (AMCL)	Agent Décisionnel
/battery_status	Topic	Publie le niveau de batterie actuel.	Simulateur Batterie	Agent Décisionnel
/charging_status	Topic	Diffuse l’état de charge (True/False).	Agent Décisionnel	Simulateur Batterie
/dock_robot	Action	Commande une manœuvre de docking de précision.	Agent Décisionnel	Nav2 (Docking Server)
/undock_robot	Action	Commande une manœuvre de retrait du dock.	Agent Décisionnel	Nav2 (Docking Server)

TABLE 2 – Interfaces de communication du système ROS 2

2.5 Conception du Module d'Intelligence Décisionnelle :

La conception du module prédictif a suivi une démarche rigoureuse de science des données, depuis la sélection des caractéristiques jusqu'à la validation du modèle.

2.5.1 Modélisation du Problème :

Le défi de savoir « quand se recharger » a été formalisé comme un problème de classification binaire supervisée. L'objectif est d'entraîner un modèle à prédire, pour un état donné du robot, l'action la plus appropriée parmi deux classes discrètes :

- **Classe 0 : EXPLORE** – L'état énergétique est suffisant pour poursuivre la mission en cours.
- **Classe 1 : DOCK** – L'état énergétique est critique ; une recharge est impérative.

2.5.2 Ingénierie des Caractéristiques (Feature Engineering) :

Le succès d'un modèle d'apprentissage dépend de manière critique de la pertinence de ses données d'entrée. Trois caractéristiques ont été soigneusement sélectionnées pour leur forte capacité prédictive :

- **battery_level** : L'indicateur le plus direct de l'état énergétique du robot.
- **path_distance_to_dock** : Le « coût en distance » pour atteindre la station de charge, calculé via le planificateur de Nav2 pour tenir compte de la topologie de l'environnement.
- **estimated_battery_for_return** : Une caractéristique synthétique combinant la distance et un modèle de consommation moyen, fournissant au modèle une estimation directe du coût énergétique du retour.

2.5.3 Logique de Contrôle de l'Agent :

L'ensemble de la logique de contrôle de l'agent, depuis l'acquisition des données jusqu'à l'actionnement, est encapsulé dans un automate fini dont le comportement est détaillé par le logigramme ci-dessous :

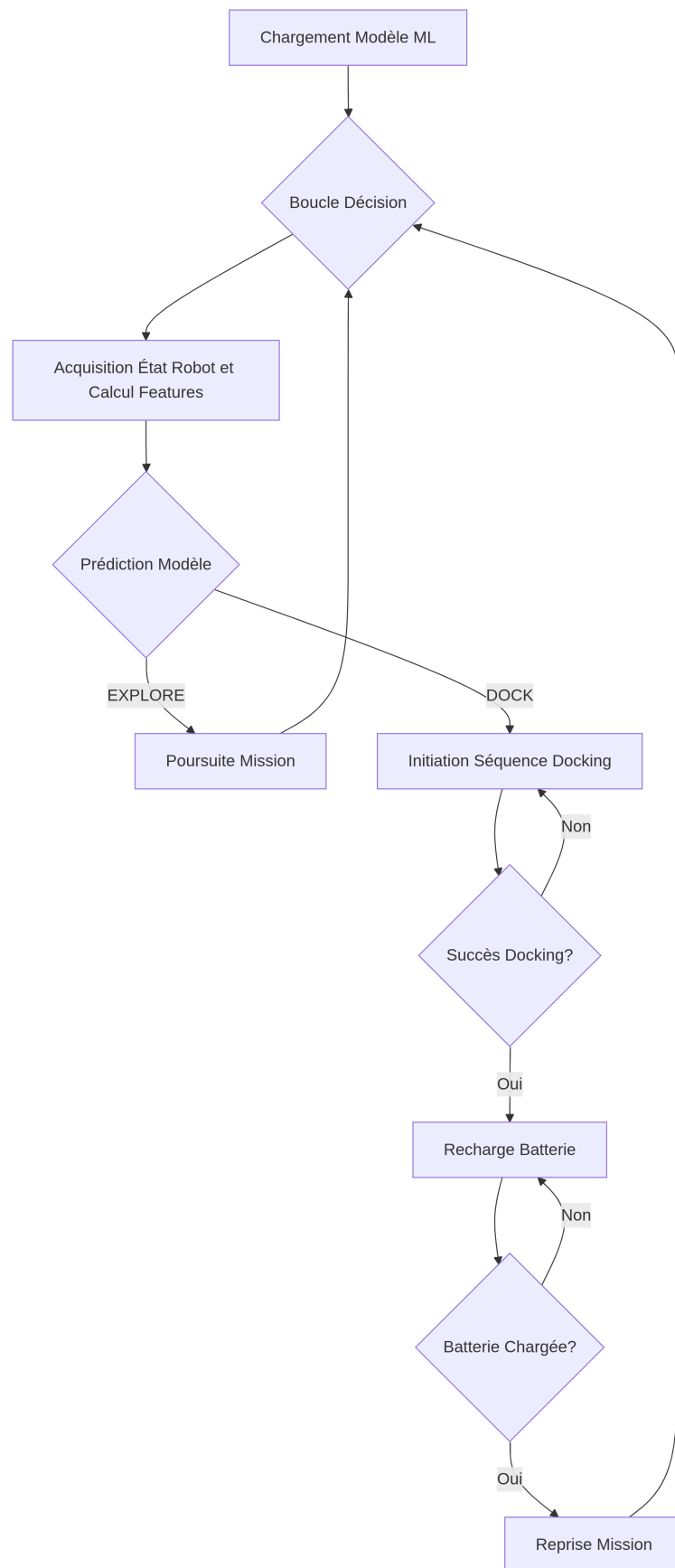


FIGURE 2 – Logigramme de l’Agent Décisionnel Autonome.

Le fonctionnement de l’agent, illustré par la figure ci-dessus, peut être décomposé en plusieurs phases clés :

- **Initialisation** : Au démarrage, l’agent charge en mémoire le modèle de Machine Learning sérialisé.
- **Boucle Principale** : À une fréquence définie, l’agent entre dans sa boucle de perception-décision-action.
- **Perception** : Il acquiert l’état actuel du système (niveau de batterie, pose du robot, etc.).
- **Inférence** : Il calcule les caractéristiques nécessaires, les normalise et interroge le modèle pour obtenir une décision (*EXPLORE* ou *DOCK*).
- **Branche “DOCK”** : Si la décision est de se recharger, l’agent préempte toute mission en cours et appelle l’action de docking spécialisée de Nav2.
- **Cycle de Recharge et Undocking** : En cas de docking réussi, l’agent attend l’événement « Batterie Pleine ». Il déclenche alors la routine de retrait avant de se mettre en attente d’une nouvelle mission.
- **Branche “EXPLORE”** : Si la décision est de continuer, l’agent s’assure que le robot poursuit sa mission actuelle.

2.5.4 Pipeline d’Apprentissage Automatique :

Le processus de développement et de déploiement du modèle d’apprentissage automatique suit un pipeline bien défini, comme illustré par le diagramme de séquence ci-dessous. Ce pipeline garantit une collecte de données structurée, un entraînement efficace du modèle et une intégration transparente dans le système de décision de l’agent.

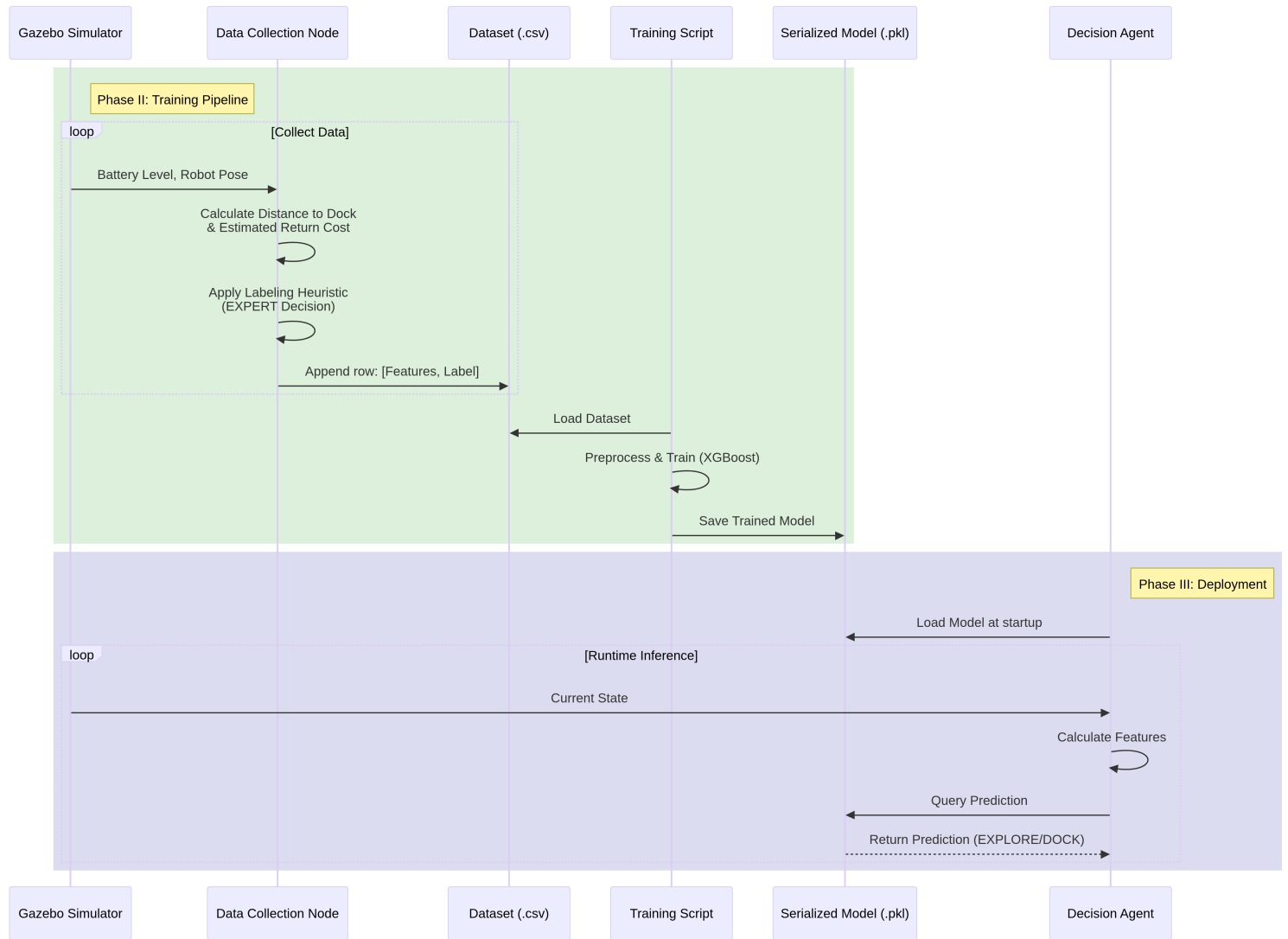


FIGURE 3 – Pipeline d’Apprentissage Automatique

Le pipeline se compose de deux phases principales :

- **Phase II : Pipeline d’Entraînement.** Cette phase est dédiée à la collecte de données et à l’entraînement du modèle. Elle implique une boucle où le simulateur Gazebo et le nœud de collecte de données travaillent ensemble pour acquérir des informations sur le niveau de batterie et la pose du robot. Une heuristique de labellisation est appliquée pour générer des décisions d’expert, qui sont ensuite utilisées pour construire un ensemble de données. Ce dataset est chargé, prétraité, et un modèle XGBoost est entraîné, puis sérialisé pour une utilisation future.
- **Phase III : Déploiement.** Cette phase concerne l’utilisation du modèle entraîné en temps réel. Au démarrage, le modèle sérialisé est chargé. Dans une boucle d’inférence, l’agent de décision calcule les caractéristiques à partir de l’état ac-

tuel du robot et interroge le modèle pour obtenir une prédiction (*EXPLORE* ou *DOCK*). Cette prédiction guide ensuite le comportement du robot.

2.6 Conclusion :

Ce chapitre a établi les fondations théoriques et conceptuelles du projet. En partant d'une analyse de l'existant, nous avons défini un ensemble de besoins précis auxquels notre architecture modulaire répond. La conception détaillée, en particulier celle du module décisionnel, a été guidée par des principes d'ingénierie logicielle et de science des données, garantissant une base solide pour la phase d'implémentation et de test qui sera décrite dans le chapitre suivant.

3 Chapitre 3 : Réalisation :

3.1 Introduction :

Ce chapitre présente la mise en œuvre pratique des choix d'architecture. Il décrit l'implémentation des différents modules logiciels, incluant la configuration de l'environnement et l'intégration système. Les paramètres critiques et les résultats de validation y sont présentés.

3.2 Environnements de travail :

3.2.1 Environnement matériel :

Le matériel à ma disposition pour le développement de mon robot est constitué d'un ordinateur portable, dont les caractéristiques sont listées dans le tableau suivant :

Ordinateur
Processeur : Intel Core i7
Mémoire : 16 GO de RAM
SE : Ubuntu 24

TABLE 3 – Environnement matériel

3.3 Choix technologiques :

Je vais à présent décrire la stack technologique complète sélectionnée pour le développement et le déploiement des fonctionnalités autonomes du robot mobile.

3.3.1 ROS2 (Robotic Operating System) :

ROS (Robot Operating System) est un système d'exploitation dédié aux robots. Il se compose d'une collection d'outils et d'algorithmes utilisés pour la programmation, la simulation et l'exécution des comportements des robots. Cette section explique les notions de bases, le mode de communication du ROS :

- Noeud "Node" : un nœud est une instance d'un exécutable. Un nœud peut correspondre à un capteur, un moteur, un algorithme de traitement, de surveillance... Chaque nœud qui se lance se déclare au Master. On retrouve ici l'architecture microkernel où chaque ressource est un nœud indépendant.
- Maître "Master" : Le Master est un service de déclaration et d'enregistrement des nœuds qui permet ainsi à des nœuds de se connaître et d'échanger de l'information.
- Comme il est mentionné ROS est basé sur la communication entre les différents nœuds, cette communication peut être établie soit de manière asynchrone via un topic ou de manière synchrone via un service.
- Un topic est un système de transport de l'information basé sur le système de l'abonnement / publication (subscribe / publish). Un ou plusieurs nœuds pourront publier de l'information sur un topic et un ou plusieurs nœuds pourront lire l'information sur ce topic.



FIGURE 4 – Exemple de communication en mode Topic

- Dans ce mode illustré par la figure 11, la communication est synchrone entre le client "client" qui demande un service concernant une tâche particulière et le serveur "server" qui est chargé de répondre aux demandes. Le serveur ne répond qu'après la demande arrivant d'un client, le client attend la réponse, c'est la raison pour laquelle ce mode est dit synchrone. Une fois que le client reçoit le message demandé, la connexion sera interrompue. Grâce à cette caractéristique, le mode service est très important pour réduire l'encombrement du réseau.
- Un message est une structure de donnée composite. Un message est composé d'une combinaison de types primitifs (chaînes de caractères, booléens, entiers, flottants...) et de message (le message est une structure récursive). Par exemple,

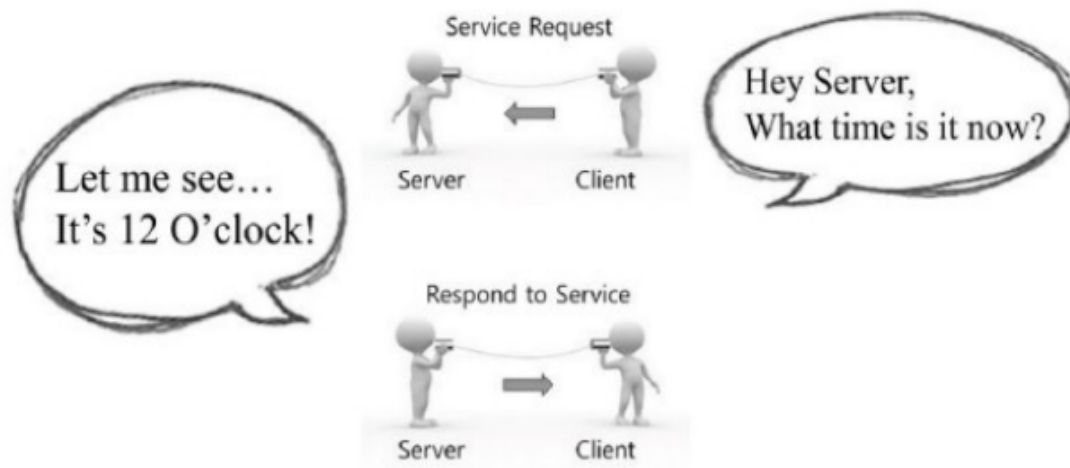


FIGURE 5 – Communication en mode Service

un nœud représentant un servomoteur du robot publiera certainement son état sur un topic (selon ce que vous aurez programmé) avec un message contenant par exemple un entier représentant la position du moteur, un flottant représentant sa température, un autre flottant représentant sa vitesse...

- Les « bags » sont des formats pour stocker et rejouer les messages échangés. Ce système permet de collecter par exemple les données mesurées par les capteurs et de les rejouer ensuite autant de fois qu'on le souhaite afin de faire de la simulation sur des données réelles. Ce système est également très utile pour déboguer un système a posteriori.

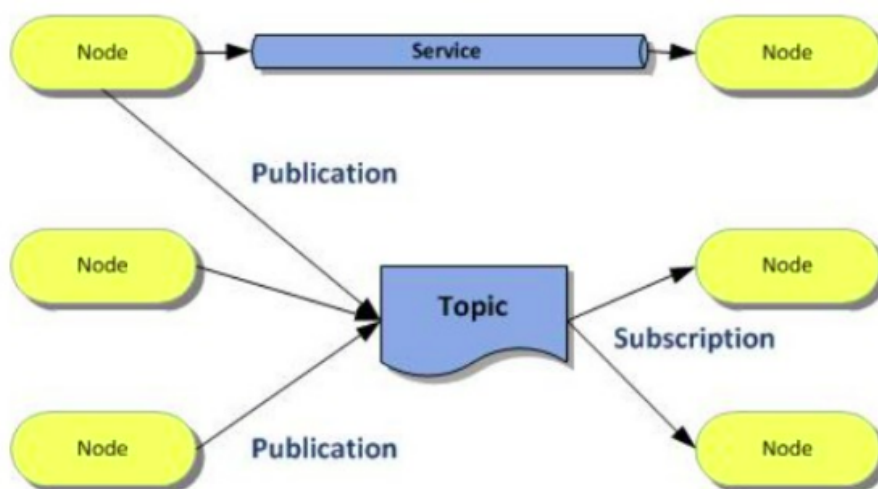


FIGURE 6 – Différents modes de communication

3.3.2 Rviz :

RViz (Ros vizualisation) est l'outil de visualisation 3D de ROS. Le but principal est de montrer les messages ROS , ce qui nous permet de vérifier visuellement les données. Par exemple, il peut visualiser la distance entre le capteur (LDS) et un obstacle, les données de nuage des points (PCD) du capteur de distance, la valeur de l'image obtenue à partir d'une caméra, et bien d'autres choses encore, sans avoir besoin de développer le logiciel séparément.



FIGURE 7 – Logo Rviz

3.3.3 Gazebo :

Gazebo offre la possibilité de simuler avec précision et efficacité des populations de robots dans des environnements intérieurs et extérieurs complexes. Au bout de vos doigts, vous disposez d'un moteur physique robuste, de graphiques de haute qualité et d'interfaces graphiques et programmatiques pratiques. Mieux encore, Gazebo est gratuit avec une communauté dynamique.



FIGURE 8 – Logo Gazebo

3.4 Fonctionnalités réalisées :

Phase I : Établissement d'une Base de Simulation et de Navigation Fiable :

Cette première phase a posé les fondations techniques nécessaires au développement de l'agent autonome. Elle a permis de créer un environnement de test robuste et de doter le robot de ses capacités de navigation et de localisation essentielles.

1. Modélisation de l'Environnement Physique :

La création d'un jumeau numérique précis du robot et de son environnement d'opération a été réalisée via le simulateur Gazebo. Cette modélisation a été cruciale pour permettre un cycle de développement et de débogage rapide et itératif, réduisant ainsi la dépendance à un hardware physique coûteux et complexe à manipuler.



FIGURE 9 – Environnement de simulation Gazebo

2. Implémentation de la Pile de Navigation Nav2 :

Le framework Nav2 a été déployé pour conférer au robot ses capacités de navigation fondamentales. Cela inclut la configuration du module de localisation AMCL (Adaptive Monte Carlo Localization) pour une estimation de pose en temps réel, ainsi que le paramétrage des serveurs de planification et de contrôle. Ces éléments sont indispensables pour permettre au robot de se déplacer de manière autonome dans un environnement connu, en évitant les obstacles et en atteignant des points de consigne.

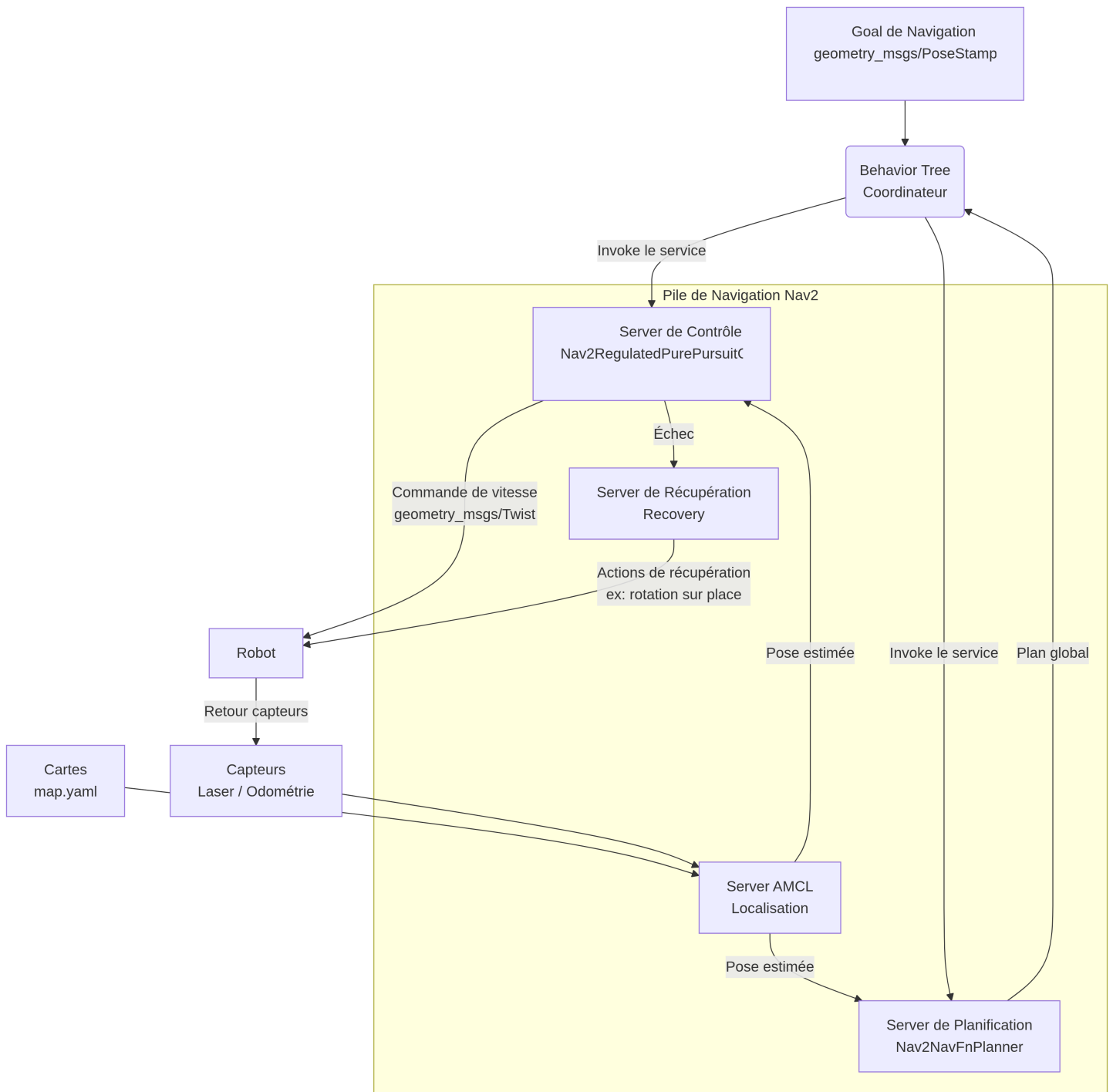


FIGURE 10 – Architecture simplifiée de la pile Nav2 et interactions principales

3. Mise en Œuvre du Docking de Précision :

Pour assurer une recharge fiable, un système de docking de précision a été mis en place. Un marqueur visuel de type AprilTag a été positionné sur la station de charge, servant de repère pour le robot. Le serveur de docking de Nav2 a été configuré pour exploiter les données de pose issues de ce marqueur, permettant ainsi des

manœuvres terminales d'approche et d'alignement très précises. Cette fonctionnalité est essentielle pour garantir que le robot puisse se connecter physiquement à sa station de recharge de manière autonome.

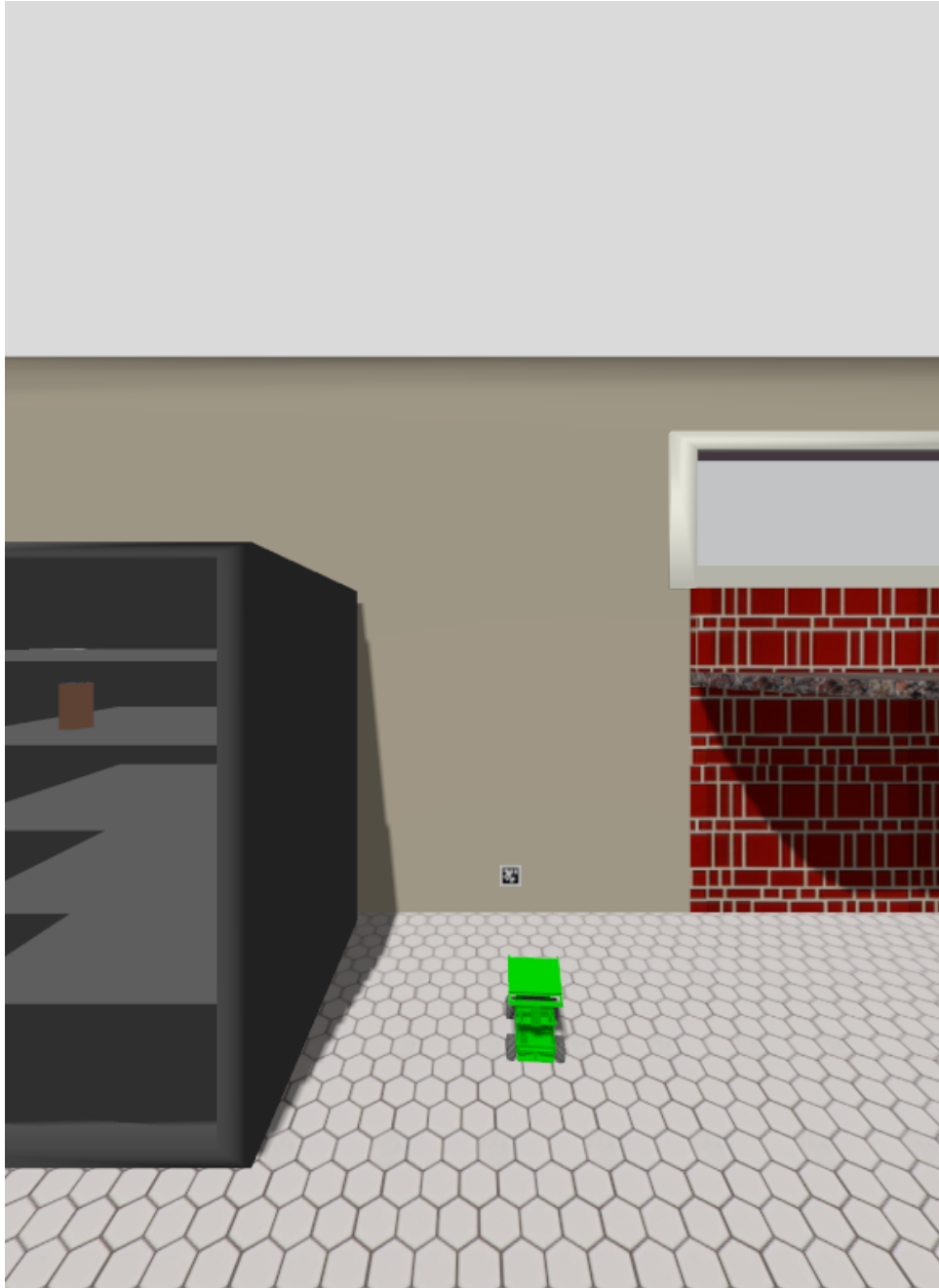


FIGURE 11 – Docking en simulation

Phase II : Conception du Modèle de Conscience Énergétique par Apprentissage Supervisé :

Cette phase représente le cœur innovant du projet, visant à doter le robot de la capacité d'anticiper ses besoins énergétiques et de prendre des décisions proactives concernant sa recharge.

1. Modélisation de la Dynamique Énergétique :

Un nœud de simulation de batterie a été développé pour modéliser de manière réaliste la décharge de la batterie en fonction de l'effort cinématique du robot. Cette modélisation dynamique fournit un état de charge qui reflète fidèlement la consommation d'énergie en fonction des mouvements et des tâches effectuées par le robot, s'éloignant ainsi des seuils de batterie fixes et statiques.

2. Constitution d'un Jeu de Données d'Entraînement :

Une campagne de collecte de données exhaustive a été menée dans l'environnement simulé. L'objectif était de capturer des vecteurs caractéristiques pertinents pour la prédiction de la nécessité de recharge. Ces vecteurs incluent le niveau de batterie actuel, la distance géodésique (distance la plus courte) jusqu'à la station de docking, et une estimation du coût énergétique du trajet de retour vers le dock. La qualité et la diversité de ce jeu de données sont fondamentales pour l'apprentissage du modèle.

1	battery_level	path_distance_to_dock	estimated_battery_for_return	decision_label
2	97.56	5.2902	16.83	0
3	95.69	5.607	17.84	0
4	93.19	6.7945	21.62	0
5	90.68	8.3359	26.52	0
6	88.98	9.8222	31.25	0
7	86.46	11.2922	35.93	0
8	84.75	12.0046	38.2	0
9	84.64	11.9805	38.12	0
10	84.64	11.9805	38.12	0
11	83.44	11.0607	35.19	0
12	82.21	9.583	30.49	0
13	81.12	7.988	25.42	0
14	78.74	7.2401	23.04	0
15	76.3	7.2682	23.13	0
16	73.85	7.8806	25.07	0
17	71.43	9.3388	29.71	0
18	69	10.4335	33.2	0
19	66.56	11.8565	37.73	0
20	65.47	12.7543	40.58	0
21	65.17	12.6112	40.13	0
22	64.35	11.8661	37.76	0
23	62.68	11.3827	36.22	0
24	60.3	11.4029	36.28	0
25	57.88	11.8945	37.85	0
26	56.1	13.053	41.53	0
27	54.95	13.0812	41.62	0
28	53.31	13.6552	43.45	0
29	50.83	14.4542	45.99	1
30	48.39	13.7445	43.73	1
31	45.97	14.7767	47.02	1
32	43.79	15.3726	48.91	1
33	43.49	15.2039	48.38	1
34	41.64	13.3903	42.61	1
35	40.19	11.9005	37.87	1
36	38.82	10.1458	32.28	0

FIGURE 12 – Extrait du jeu de données d'entraînement

3. **Annotation Automatisée (Labeling)** : Chaque vecteur de données collecté a été automatiquement annoté avec une « décision experte » (EXPLORE ou DOCK). Cette annotation a été basée sur une heuristique prédéfinie, prenant en compte l'autonomie résiduelle du robot. Ce processus de labellisation est crucial pour l'apprentissage supervisé, car il fournit au modèle les exemples de décisions à apprendre.

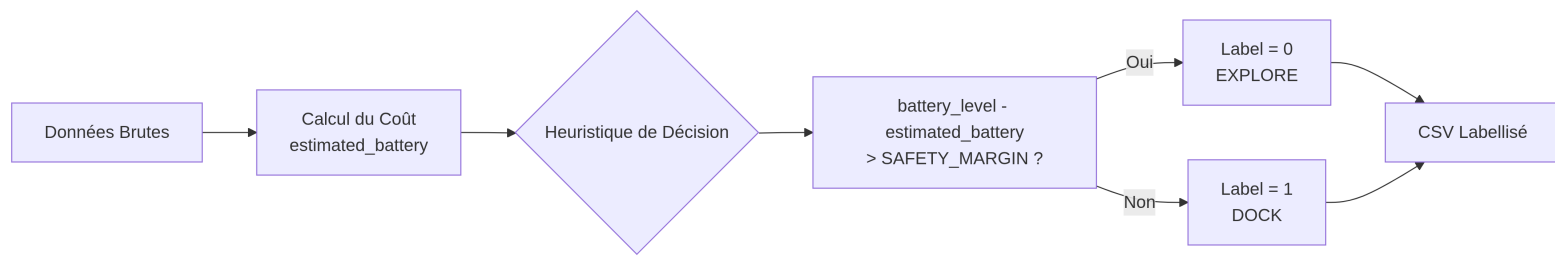


FIGURE 13 – Processus d’annotation

4. Entraînement et Validation du Modèle

Pour la prise de décision autonome du robot, un modèle prédictif a été développé en utilisant l’algorithme **XGBoost**, une référence reconnue pour sa robustesse et sa précision en classification. L’objectif de ce modèle est de choisir l’action la plus pertinente (**EXPLORE** ou **DOCK**) en fonction de l’état actuel du robot.

- Validation des Performances :

Après son entraînement, le modèle a été soumis à une phase de validation rigoureuse sur un jeu de données de test. Les résultats obtenus sont excellents, avec une **précision globale (accuracy) de 96.6%**, démontrant sa très grande fiabilité. La matrice de confusion ci-dessous révèle un très faible nombre d’erreurs de classification.

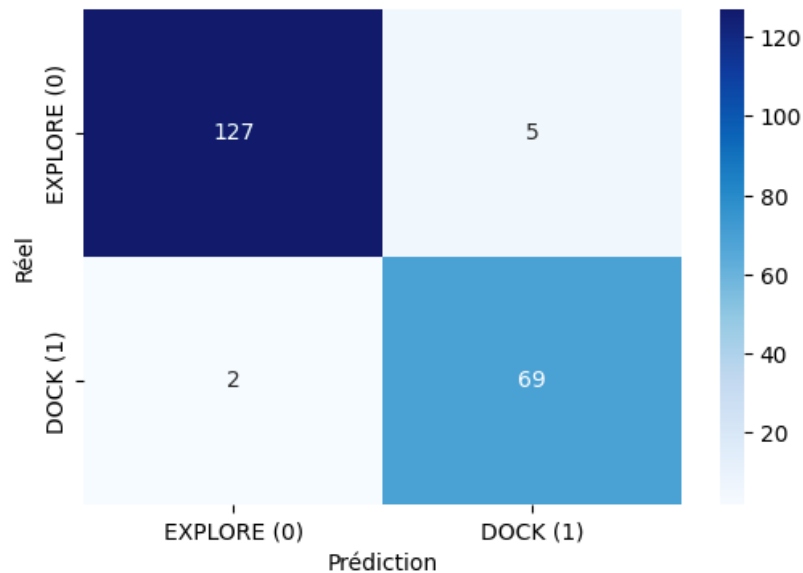


FIGURE 14 – Matrice de confusion du modèle sur le jeu de test.

Comme le montre le tableau ci-dessous, le rapport de classification confirme la haute performance du modèle. Les scores F1 sont particulièrement élevés, avec

0,97 pour EXPLORE et 0,95 pour DOCK.

Classe	Précision	Rappel	F1-Score	Support
EXPLORE (0)	0.98	0.96	0.97	132
DOCK (1)	0.93	0.97	0.95	71

TABLE 4 – Rapport de classification détaillé par classe.

- **Interprétabilité du Modèle :** Comme l'illustre la figure ci-dessous , L'analyse de l'importance des caractéristiques montre que la décision du modèle repose sur des critères logiques et pertinents. Le **niveau de la batterie (battery_level)** est, de loin, le facteur le plus déterminant, suivi par la **distance au dock(path_distance_to_dock)**. Cette hiérarchie confirme que le modèle a bien appris la règle de gestion fondamentale du robot.

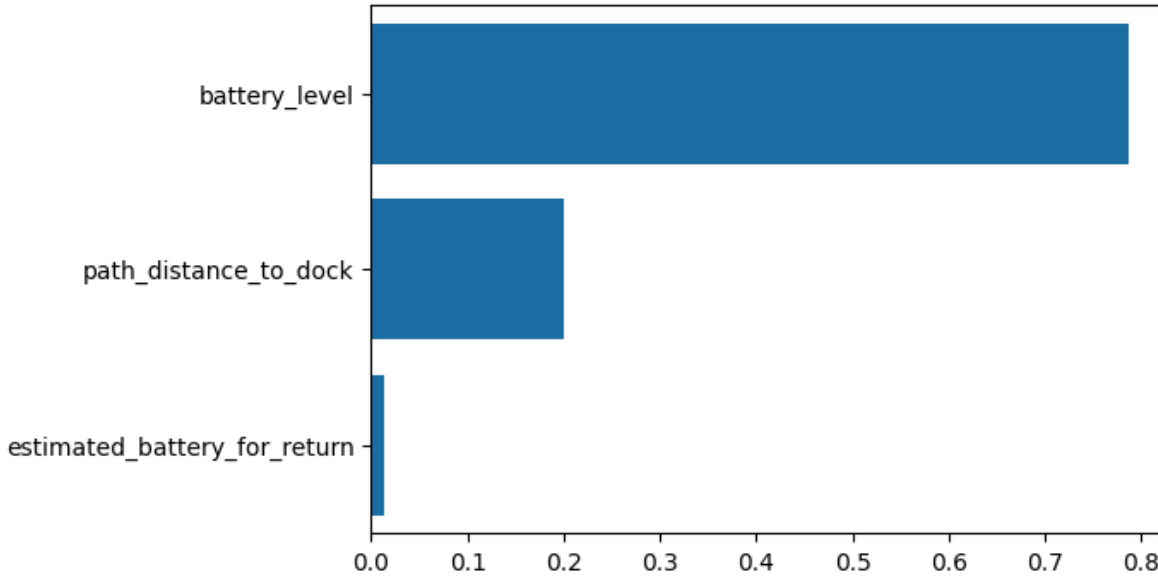


FIGURE 15 – Importance des caractéristiques pour la prédiction du modèle.

Phase III : Intégration du Modèle et Architecture Décisionnelle :

Cette phase a consisté à intégrer l'intelligence prédictive développée en Phase II au sein de l'architecture de contrôle du robot, transformant ainsi la prédiction en action.

1. **Développement de l'Agent Décisionnel :** Un nœud ROS 2 central a été développé, agissant comme le « cerveau exécutif » du robot. Ce nœud est responsable du chargement du modèle de Machine Learning sérialisé et de son exécution.

Il constitue le point névralgique où les données du robot sont traitées pour générer des décisions comportementales.

2. **Implémentation de la Boucle de Décision :** À une fréquence définie, le nœud de l'agent décisionnel perçoit l'état actuel du robot (niveau de batterie, position, mission en cours). Ces informations sont ensuite transmises au modèle de Machine Learning qui fournit une prédiction (EXPLORE ou DOCK). En fonction de cette prédiction, l'agent agit en conséquence : si la décision est DOCK, il interrompt la navigation en cours et initie la procédure de docking ; sinon, il permet au robot de poursuivre sa mission. Cette boucle garantit une adaptation dynamique du comportement du robot à son état énergétique.

Phase IV : Réalisation de la Boucle d'Autonomie Complète :

La dernière phase du projet a été dédiée à la fermeture de la boucle d'autonomie, assurant ainsi un cycle complet de mission, recharge et reprise d'activité pour le robot.

1. **Gestion du Cycle de Recharge :** Le simulateur de batterie, développé en Phase II, a été intégré pour augmenter l'état de charge du robot lorsque celui-ci est correctement docké à sa station. Cette simulation réaliste permet de valider le processus de recharge et son impact sur l'autonomie du robot.
2. **Automatisation du Détachement :** Une fois que le robot a atteint un niveau de charge complet (ou un seuil prédéfini de charge suffisante), l'agent décisionnel déclenche automatiquement la manœuvre de retrait de la station de docking. Cette étape est cruciale pour que le robot puisse reprendre ses missions sans intervention humaine.
3. **Réinitialisation du Cycle Comportemental :** Après s'être détaché et avoir quitté la station de recharge, le robot revient à un état nominal, prêt à accepter de nouvelles missions. Le cycle complet, englobant la mission initiale, la décision de recharge, le docking, la recharge elle-même, le détachement et la reprise de mission, est ainsi bouclé. Cette fonctionnalité garantit une persistance opérationnelle à long terme pour le robot mobile.

3.5 Conclusion :

Ce chapitre a concrétisé les choix architecturaux en un robot autonome doté d'une conscience énergétique. De la simulation à l'intégration du modèle prédictif, chaque phase a contribué à une gestion proactive de l'autonomie. Le projet valide ainsi une persistance opérationnelle complète, ouvrant la voie à des applications robotiques avancées.

Conclusion générale :

Ce projet a permis le développement et la validation d'un système d'autonomie énergétique pour robot mobile, mettant en œuvre une approche innovante de gestion proactive des ressources énergétiques. La solution développée intègre avec succès des techniques d'apprentissage automatique au sein d'une architecture ROS 2, démontrant des performances significativement supérieures aux méthodes traditionnelles basées sur des seuils fixes.

Les résultats obtenus confirment la validité de l'approche proposée et une architecture système garantissant une coordination optimale entre les différents modules. La validation exhaustive en environnement simulé a permis de vérifier la robustesse et la fiabilité de l'ensemble de la solution.

Sur le plan technique, ce travail a nécessité la maîtrise et l'intégration de plusieurs domaines de compétences : modélisation énergétique dynamique, apprentissage automatique, architecture logicielle temps réel et systèmes embarqués. L'approche modulaire adoptée assure la maintenabilité et l'évolutivité de la solution.

D'un point de vue professionnel, ce projet a offert l'opportunité de développer une expertise approfondie en conception de systèmes robotiques autonomes et en intégration de technologies d'intelligence artificielle. Les méthodologies employées, depuis la modélisation jusqu'aux tests de validation, respectent les standards industriels en vigueur.

Ce travail ouvre plusieurs voies de développement futures, notamment le passage à la plateforme physique réelle, l'amélioration des performances temps réel et le renforcement des capacités d'adaptation contextuelle. Il constitue une base solide pour l'exploration de méthodes avancées en intelligence artificielle appliquée à la robotique mobile.

Cette réalisation contribue au développement de systèmes robotiques plus autonomes et plus efficaces, tout en démontrant la maturité des solutions open-source pour des applications industrielles exigeantes. Elle représente une avancée concrète vers une robotique véritablement intelligente et capable de gérer de manière autonome ses ressources opérationnelles.

Mon bilan de compétences :

Quels métiers d'ingénieurs » j'ai l'intention d'exercer dans le futur ?

Je souhaite poursuivre ma carrière dans le domaine de la robotique, en me concentrant sur la conception et le développement de systèmes robotiques avancés. Mon ambition est de participer activement à l'innovation technologique, en intégrant des solutions novatrices et en explorant de nouvelles applications pour répondre aux défis futurs.

Quels métiers d'ingénieurs » je ne souhaiterai pas exercer ?

Je ne souhaiterais pas exercer des métiers d'ingénieur liés au matériel électrique ou à la conception matérielle, préférant me concentrer sur les aspects logiciels et la programmation des systèmes robotiques.

Fiche bilan de compétences

	Compétences totalement ou partiellement acquises			Compétences à développer en première priorité		Compétences à développer en deuxième priorité	
	La compétence	Degré de maîtrise (en %)	Occasion de pratique	La compétence ciblée	Moyen à utiliser	La compétence ciblée	Moyen à utiliser
Compétences techniques pratiques	Utilisation de ROS 2	80%	stage	Déploiement ML temps réel sous ROS 2		Optimisation performances ROS 2	
	ML	70 %	stage				
Compétences Socioprofessionnelles (soft skills)							
	Communication	80 %					
	Résolution de problèmes	80 %					

Références

- ROS 2 Documentation – <https://docs.ros.org/>
- Navigation2 (Nav2) – <https://navigation.ros.org/>
- Gazebo Tutorials – <https://gazebo.org/docs>
- XGBoost Documentation – <https://xgboost.readthedocs.io/>
- OpenCV Documentation – <https://docs.opencv.org/>
- Industrial Training – <https://industrial-training-master.readthedocs.io/>
- AprilTag Detection – <https://github.com/AprilRobotics/apriltag>
- *Mastering ROS for Robotics Programming* (Lentin Joseph, 2021)
- *Robot Operating System for Absolute Beginners* (Lentin Joseph, 2022)